AGENTOS - A HYBRID MULTI-AGENT RUNTIME ENVIRONMENT FOR AUTONOMOUS AI APPLICATIONS

A PROJECT REPORT

Submitted by

GOGUL A (2023115020)

MOHAN VISHNU KUMAR M (2023115026)

OVIYASHREE C J (2023115052)

KUDIMI GUNA SEKHAR (2023115126)

submitted to the Faculty of

INFORMATION SCIENCE AND TECHNOLOGY

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY



DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY

COLLEGE OF ENGINEERING, GUINDY

ANNA UNIVERSITY

CHENNAI 600 025

OCTOBER 2026

ANNA UNIVERSITY

CHENNAI - 600 025

BONA FIDE CERTIFICATE

Certified that this project report titled **AgentOS - A Hybrid Multi-Agent Runtime Environment for Autonomous AI Applications** is the bona fide work of Gogul A (2023115020), Mohan Vishnu Kumar M (2023115026), Oviyashree C J (2023115052), and Kudimi Guna Sekhar (2023115126), who carried out the project work under my supervision. Certified further that, to the best of my knowledge and belief, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or an award was conferred on an earlier occasion on this or any other candidate.

PLACE:
DATE:

Dr. SELVI RAVINDRAN
ASSOCIATE PROFESSOR
PROJECT GUIDE
DEPARTMENT OF IST, CEG
ANNA UNIVERSITY
CHENNAI 600025

COUNTERSIGNED

Dr. P. YOGESH
HEAD OF THE DEPARTMENT
DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY
COLLEGE OF ENGINEERING, GUINDY
ANNA UNIVERSITY
CHENNAI 600025

ABSTRACT

Large-language-model applications are evolving from single-turn chat interfaces into systems that plan, use tools, preserve context, and coordinate multiple specialised agents. However, application developers commonly rebuild scheduling, memory, event propagation, execution monitoring, and failure handling for each project. This report presents AgentOS, a hybrid multi-agent runtime environment that provides these operating-system-style services around autonomous AI applications. The implemented system combines a Next.js user interface, a FastAPI gateway, a Redis-backed event path, MongoDB task history, ChromaDB semantic retrieval, structured LLM planning, dependency-aware directed acyclic graph execution, semantic context paging, a global priority-aging scheduler, token and concurrency admission control, and a sandboxed Python code runner with bounded self-healing retries. The frontend exposes the generated plan graph and live scheduler and execution activity through WebSocket updates. The report distinguishes implemented facilities from proposed components such as complete role-based access control, distributed locking, hybrid model routing, and durable coroutine restoration after a kernel restart. Verification is based on repository compile checks, TypeScript checks, and focused smoke tests rather than unsubstantiated performance claims. AgentOS therefore provides a modular foundation for studying reliable, observable, and resource-aware multi-agent execution.

Keywords: multi-agent runtime, semantic planning, DAG execution, context paging, priority scheduling.

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, **Dr. Selvi Ravindran**, Associate Professor, Department of Information Science and Technology, Anna University, College of Engineering, Guindy, for her guidance, technical suggestions, and continuous encouragement throughout this work.

We thank the faculty members and staff of the Department of Information Science and Technology for providing the academic environment and facilities required to undertake this project. We also acknowledge our classmates, friends, and families for their support during the design, implementation, testing, and documentation of AgentOS.

Gogul A
Mohan Vishnu Kumar M
Oviyashree C J
Kudimi Guna Sekhar

TABLE OF CONTENTS

ABSTRACT iii
ACKNOWLEDGEMENT iv
LIST OF TABLES viii
LIST OF FIGURES ix
LIST OF SYMBOLS AND ABBREVIATIONS x

1 INTRODUCTION **1**
1.1 BACKGROUND 1
1.2 MOTIVATION 1
1.3 PROBLEM STATEMENT 2
1.4 OBJECTIVES 2
1.5 SCOPE 3
1.6 CONTRIBUTIONS 3
1.7 REPORT ORGANIZATION 3

2 LITERATURE SURVEY **4**
2.1 OVERVIEW 4
2.2 AGENT RUNTIMES AND AUTONOMOUS AGENTS 4
2.3 MEMORY AND CONTEXT MANAGEMENT 5
2.4 PLANNING, TOOL USE, AND REFLECTION 5
2.5 SCHEDULING, RESOURCES, AND RELIABILITY 6
2.6 SECURITY AND SANDBOXED EXECUTION 7
2.7 EVENT-DRIVEN INFRASTRUCTURE 7
2.8 RESEARCH GAP AND POSITIONING 7

3 SYSTEM DESIGN **9**
3.1 DESIGN PRINCIPLES 9
3.2 OVERALL ARCHITECTURE 9
3.3 FRONTEND, GATEWAY, AND EVENT FLOW 11
3.4 PLANNER AND DAG EXECUTION 11
3.5 CONTEXT PAGING 13
3.6 GLOBAL SCHEDULER AND RESOURCE MANAGER 13
3.7 RESOURCE MANAGEMENT 14
3.8 TOOL MANAGEMENT 15

3.9 AGENT LIFECYCLE AND EVENTS 16
3.10 SANDBOXED CODE RUNNER 17
3.11 IMPLEMENTED AND PROPOSED BOUNDARIES 18

4 IMPLEMENTATION 19
4.1 IMPLEMENTATION ENVIRONMENT 19
4.2 SEMANTIC AGENT RETRIEVAL 19
4.3 PLANNER AND FALLBACK 20
4.4 DAG VALIDATION 21
4.5 PRIORITY-AGING SCHEDULER 22
4.6 RESOURCE ADMISSION AND TOKEN BUDGETS 23
4.7 REDIS EVENTS AND BACKEND FORWARDING 23
4.8 CONTEXT PAGE-OUT AND PAGE-IN 24
4.9 SANDBOXED EXECUTION AND SELF-HEALING 25
4.10 FRONTEND OBSERVABILITY 25
4.11 CONFIGURATION AND VERIFICATION 25
4.12 AGENTOS ALGORITHMIC SPECIFICATION 26

5 EVALUATION FRAMEWORK AND METRICS 33
5.1 EVALUATION NOTATION 33
5.2 PLANNING QUALITY METRICS 33
5.3 EXECUTION RELIABILITY METRICS 35
5.4 SCHEDULER AND RESOURCE METRICS 35
5.5 CONTEXT PAGING AND RETRIEVAL METRICS 36
5.6 EVENT AND OBSERVABILITY METRICS 37
5.7 CODE RUNNER SAFETY METRICS 37
5.8 REPORTING PROTOCOL 38

6 SUMMARY AND FUTURE WORK 40

7 AGENT CATALOGUE AND INTERNAL WORKFLOWS 41

AGENT CATALOGUE AND INTERNAL WORKFLOWS 41
7.1 Agent A1: System Information Agent 42
7.2 Agent A2: Date and Time Agent 43
7.3 Agent A3: Research Agent 44
7.4 Agent A4: File and Document Agent 45

7.5 Content Creator Agent 46
7.6 Email Agent 47
7.7 Weather Agent 48
7.8 Loan EMI Calculator Agent 49
7.9 Crypto PnL Agent 50
7.10 Code Runner Agent 51
7.11 Agent Creator 52

REFERENCES **53**

LIST OF TABLES

3.1 Implemented and future AgentOS modules 18

4.1 Principal Redis channels and keys 24
4.2 Selected AgentOS configuration values 26

5.1 Metrics for evaluating planner quality 34
5.2 Metrics for execution reliability 35
5.3 Metrics for scheduling and resource admission 36

LIST OF FIGURES

3.1	Overall AgentOS system architecture	10
3.2	Frontend, FastAPI gateway, Redis channels, AI kernel, and agent-workforce flow	11
3.3	Planner, validation, and dependency-aware execution flow	12
3.4	Context page-out, vector-store retrieval, and semantic page-in flow	13
3.5	Global agent scheduler, lifecycle events, resource checks, and runtime feedback	14
3.6	Resource-manager decision flow supplied for the AgentOS design	15
3.7	Tool selection, permission checking, invocation, and result return	16
3.8	Agent lifecycle transitions	17
3.9	Sandboxed Python execution and bounded self-healing loop	17
4.1	Optimized semantic planner and DAG construction	21
7.1	AgentOS agent catalogue	41
7.2	A1 system-information logic	42
7.3	A2 date-and-time logic	43
7.4	A3 research and source-verification logic	44
7.5	A4 file-operation and deletion-confirmation logic	45
7.6	Content Creator drafting logic	46
7.7	Email delivery and simulation-mode logic	47
7.8	Weather retrieval and reporting logic	48
7.9	Loan EMI and amortisation logic	49
7.10	Cryptocurrency PnL and ROI logic	50
7.11	Code generation, sandbox execution, and repair logic	51
7.12	Dynamic agent creation and registration logic	52

LIST OF SYMBOLS AND ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
DAG	Directed Acyclic Graph
FSM	Finite-State Machine
IPC	Inter-Process Communication
LLM	Large Language Model
MCP	Model Context Protocol
MongoDB	Document-oriented database used for task history and metadata
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
Redis	In-memory data store used for queues and event transport
UI	User Interface
WebSocket	Bidirectional communication protocol over a persistent connection
α	Aging factor used in effective-priority calculation
$P_{\max}$	Maximum effective priority
T_q	Cooperative dispatch time quantum

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Large-language-model (LLM) applications have progressed from reactive question-answering interfaces to systems that can plan, call tools, maintain memory, and delegate work to specialised agents. A useful multi-agent application therefore needs more than a prompt and a model endpoint. It needs a runtime that can represent work, enforce dependencies, manage limited provider and execution resources, preserve useful context, report progress, and recover safely from failures.

Operating-system concepts provide a useful design analogy. A task is submitted to a gateway, a planner decomposes it into executable steps, a scheduler selects runnable agent work, a resource manager performs admission control, and an event path exposes state transitions. AgentOS applies this analogy to LLM-powered agents while retaining the uncertainty and latency of model calls.

1.2 MOTIVATION

In a conventional chatbot, application logic, conversation state, tool invocation, and error handling are often tightly coupled. This makes it difficult to run independent work concurrently or to explain why an agent was selected. Long conversations also exceed the practical context window of a model, while blindly truncating history can remove information needed by later steps. Finally, direct code execution or unrestricted tools create safety and observability risks.

AgentOS is motivated by the need for a reusable runtime layer. The project investigates how planning, agent retrieval, DAG execution, semantic context paging, global scheduling, resource accounting, event streaming, and sandboxed execution can be composed into one observable system.

1.3 PROBLEM STATEMENT

There is no single lightweight runtime in the project environment that coordinates heterogeneous agents across multiple concurrent tasks while enforcing dependencies, observing token and concurrency budgets, preserving relevant context, and exposing execution state to a user interface. Reimplementing these facilities inside each application increases duplication, makes failures difficult to diagnose, and prevents systematic evaluation of scheduling and resource policies.

1.4 OBJECTIVES

The objectives of AgentOS are:

1. Develop a modular AI-kernel-style runtime for specialised agents;
2. Generate structured execution plans, retrieve suitable agents semantically, and validate their dependencies as a DAG;
3. Manage bounded conversational context through semantic page-out and page-in using a vector-backed swap space;
4. Schedule runnable work with priority aging, fairness, cooperative dispatch, concurrency limits, and token budgets; and
5. Execute agent and code-runner tasks reliably while publishing lifecycle events and measurable activity to the user interface.

1.5 SCOPE

The current phase covers a Next.js frontend, FastAPI gateway, Python kernel, Redis event transport, MongoDB task persistence, ChromaDB retrieval, structured planning, DAG validation and Mermaid generation, context paging, a global in-process scheduler, resource admission, and a sandboxed Python runner. The scope does not claim complete RBAC, distributed scheduler workers, forced termination of active LLM calls, automatic coroutine restoration, or a production-grade hybrid model router. These are documented as future work.

1.6 CONTRIBUTIONS

The project contributions are a unified execution model for heterogeneous agents, a visible plan graph, a Redis-mediated activity stream, a semantic context paging mechanism, a priority-aging scheduler with resource admission, and a bounded code-execution/self-healing path. The implementation also makes a deliberate distinction between design aspirations and features that are present and testable in the repository.

1.7 REPORT ORGANIZATION

Chapter 2 surveys research and technical systems related to autonomous agents, memory, planning, scheduling, tool use, security, event transport, and vector retrieval. Chapter 3 presents the AgentOS architecture and data flows. Chapter 4 describes the implementation algorithms, storage conventions, events, and verification evidence. The conclusion records current limitations and future extensions.

CHAPTER 2

LITERATURE SURVEY

2.1 OVERVIEW

This chapter reviews work that informs the design of AgentOS. The survey combines peer-reviewed or archival research with official technical documentation for infrastructure components. Each group is considered in terms of its contribution, limitation for the present project, and the way AgentOS uses or extends the idea.

2.2 AGENT RUNTIMES AND AUTONOMOUS AGENTS

AIOS frames an LLM agent operating system around scheduling, context management, memory, storage, access control, and tool management [1]. Its operating-system analogy directly motivates the AgentOS kernel, although the current AgentOS implementation is smaller and uses a Python/Redis/MongoDB stack rather than a complete OS abstraction. The broader survey by Wang et al. classifies LLM-based agents by construction, applications, and evaluation [2]; it identifies coordination and reliability as open challenges that AgentOS addresses through explicit events and DAG dependencies.

The LLM-as-an-OS proposal treats models as system resources and agents as applications [3]. AutoGen demonstrates programmable multi-agent conversation and human-in-the-loop patterns [4], while LangGraph provides graph-based orchestration and durable execution concepts [5]. These systems

show the value of explicit state and orchestration, but do not provide the exact global priority-aging and token-admission policy implemented here.

2.3 MEMORY AND CONTEXT MANAGEMENT

MemGPT introduces virtual context management that moves information between active context and external storage [6]. MemoryOS organises memory into short-, mid-, and long-term layers and evaluates memory retention for conversational agents [7]. These works motivate AgentOS context page-out and page-in. AgentOS uses semantic embeddings in ChromaDB and an active-window policy; it currently provides a project-level paging mechanism rather than a learned memory hierarchy or an independently benchmarked memory score.

Retrieval-Augmented Generation (RAG) combines generation with a non-parametric knowledge store [8]. ChromaDB documents collections, embeddings, and distance-based retrieval [9]. AgentOS uses this pattern both for agent capability retrieval and for swapped context. The limitation is that retrieval quality depends on the embedding model and distance configuration; the current implementation exposes similarity metadata but does not claim domain-wide retrieval benchmarks.

2.4 PLANNING, TOOL USE, AND REFLECTION

ReAct interleaves reasoning and action so that an agent can use external tools during problem solving [10]. Toolformer studies self-supervised learning for deciding when and how to call APIs [11]. Generative Agents combine memory, reflection, and planning for believable behaviour [12]. Reflexion uses verbal feedback to improve subsequent attempts without

updating model weights [13]. AgentOS adopts structured planning, explicit tool boundaries, persisted events, and bounded self-healing, but does not train models or claim the autonomous reflection capabilities of these research systems.

The Transformer architecture remains the dominant basis for modern LLMs [14]. Its context limitation makes external context management relevant to AgentOS, while the model itself remains an external provider selected by the planner/runtime.

2.5 SCHEDULING, RESOURCES, AND RELIABILITY

Classical operating-system scheduling shows why priority alone can starve old work and why aging can improve fairness [15]. Linux cgroups provide a practical model for resource accounting and isolation [16]. AgentOS adapts these ideas to agent work: effective priority increases with queue wait, admission checks global and per-task slots, and token reservations are tracked before dispatch. Unlike a CPU scheduler, AgentOS cannot forcibly preempt an in-flight remote LLM request; its quantum is cooperative and checkpoint-based.

The workflow and orchestration literature emphasises explicit dependency graphs and stateful execution. LangGraph persistence documents checkpointed graph state [17], while Temporal documents durable workflow execution [18]. AgentOS currently validates and executes a DAG in-process and persists task/event evidence; automatic continuation of interrupted coroutines remains future work.

2.6 SECURITY AND SANDBOXED EXECUTION

The Model Context Protocol specifies a standard way for AI applications to connect models with tools and data sources [19]. OWASP's LLM security guidance highlights risks including excessive agency, insecure tool use, and code execution [20]. Python subprocess execution and filesystem scoping are therefore bounded in the code runner. Firecracker demonstrates the stronger isolation achievable with microVMs [21]. AgentOS's current runner limits duration and output and scopes its working directory, but it is not a substitute for a hardened kernel or microVM sandbox.

2.7 EVENT-DRIVEN INFRASTRUCTURE

Redis documents Pub/Sub as a low-latency message distribution mechanism [22]. MongoDB provides durable document storage suitable for task history and metadata [23]. AgentOS uses Redis for transient task/event transport and queue metadata and MongoDB for durable task records. Redis Pub/Sub alone is not a durable event log; recovery therefore persists relevant state in Redis hashes and MongoDB events, with full replay and exactly-once semantics left for future work.

2.8 RESEARCH GAP AND POSITIONING

The reviewed systems provide strong solutions for individual concerns: planning, memory, graph orchestration, tool use, security, or workflow durability. AgentOS's contribution is an integrated educational/runtime prototype that makes these concerns observable together in one project. Its distinctive implementation choices are semantic agent selection, explicit DAG validation, global priority-aging scheduling,

token/concurrency admission, Redis lifecycle events, context paging, and frontend graph/activity observability. The current phase prioritises a transparent and testable architecture over claims of production isolation or benchmark leadership.

CHAPTER 3

SYSTEM DESIGN

3.1 DESIGN PRINCIPLES

AgentOS is designed as a modular runtime. The gateway handles external requests, the orchestrator converts a prompt into a structured plan, and the kernel owns execution concerns. Redis is used for low-latency coordination and event transport, while MongoDB stores task history. ChromaDB stores embeddings used for semantic agent retrieval and context swap. The design keeps the current dependency truth in the DAG executor and treats scheduler decisions as admission and dispatch decisions.

3.2 OVERALL ARCHITECTURE

As shown in Figure 3.1, AgentOS is organised as an overall runtime architecture connecting the user interface, gateway, planner, kernel, storage, tools, agents, and model engines. The figure combines implemented runtime components with architectural modules that are planned or only partially implemented; their implementation status is documented separately in Section 3.11. A user submits a task through the Next.js interface. The FastAPI gateway creates or retrieves task state and communicates with the kernel. The planner selects agents and creates steps; the kernel validates and executes those steps while publishing activity events.

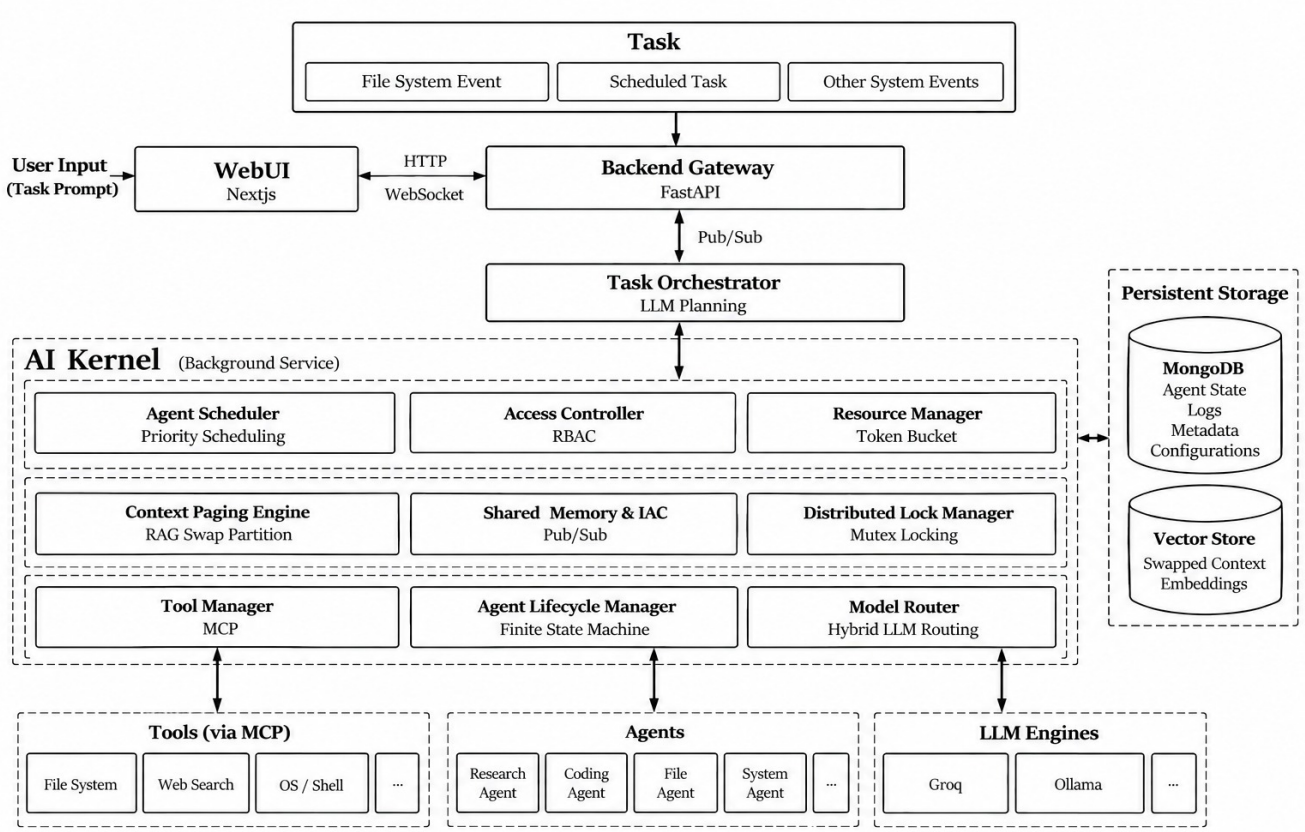


Figure 3.1: Overall AgentOS system architecture

3.3 FRONTEND, GATEWAY, AND EVENT FLOW

The frontend sends task prompts over HTTP and receives live activity over WebSocket. The backend listener subscribes to the kernel event channel and forwards events to connected clients. Figure 3.2 shows the separation between the task submission channel and the kernel event channel. MongoDB persistence is performed by the kernel/repository layer so the UI can later query task history.

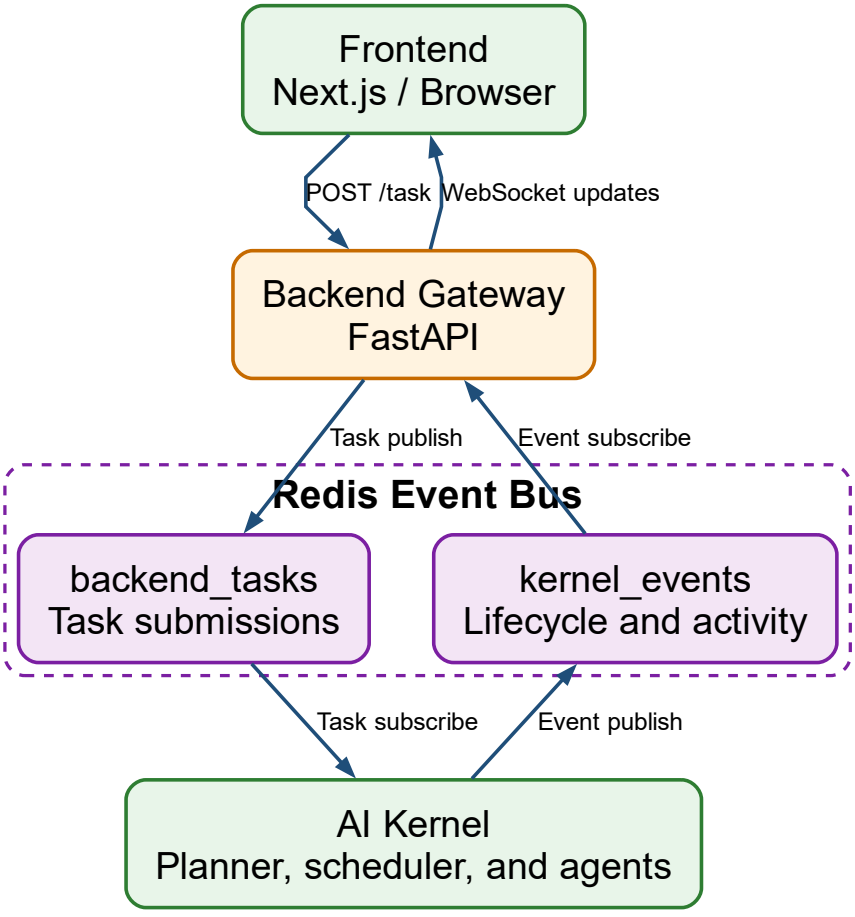


Figure 3.2: Frontend, FastAPI gateway, Redis channels, AI kernel, and agent-workforce flow

3.4 PLANNER AND DAG EXECUTION

The planner retrieves relevant agent descriptions, requests a structured execution plan, and applies a fallback plan when the provider is unavailable. Each step includes an identifier, agent name, description, and dependency list. The validator canonicalises identifiers, rejects unknown dependencies and cycles, and supplies a sequential fallback only when a plan omits dependencies. The executor runs only steps whose prerequisites are complete. Figure 3.3 summarises this path. The frontend uses the same validated plan to generate Mermaid source and draw the plan graph, ensuring that displayed edges represent dependencies rather than decorative links.

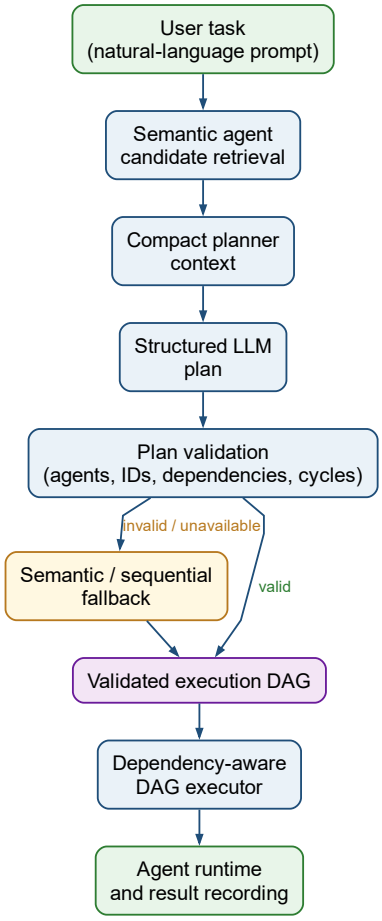


Figure 3.3: Planner, validation, and dependency-aware execution flow

3.5 CONTEXT PAGING

The context pager keeps a bounded active message window. When the window is exceeded, older or less relevant messages are embedded and stored in the ChromaDB swap collection. A later query retrieves semantically related pages and injects them into the active context. ChromaDB's cosine space returns distance; the implementation converts it as $s = 1 - d$, where d is the returned cosine distance. Figure 3.4 presents the page-out and page-in flow. This mechanism reduces active prompt growth but does not guarantee perfect recall.

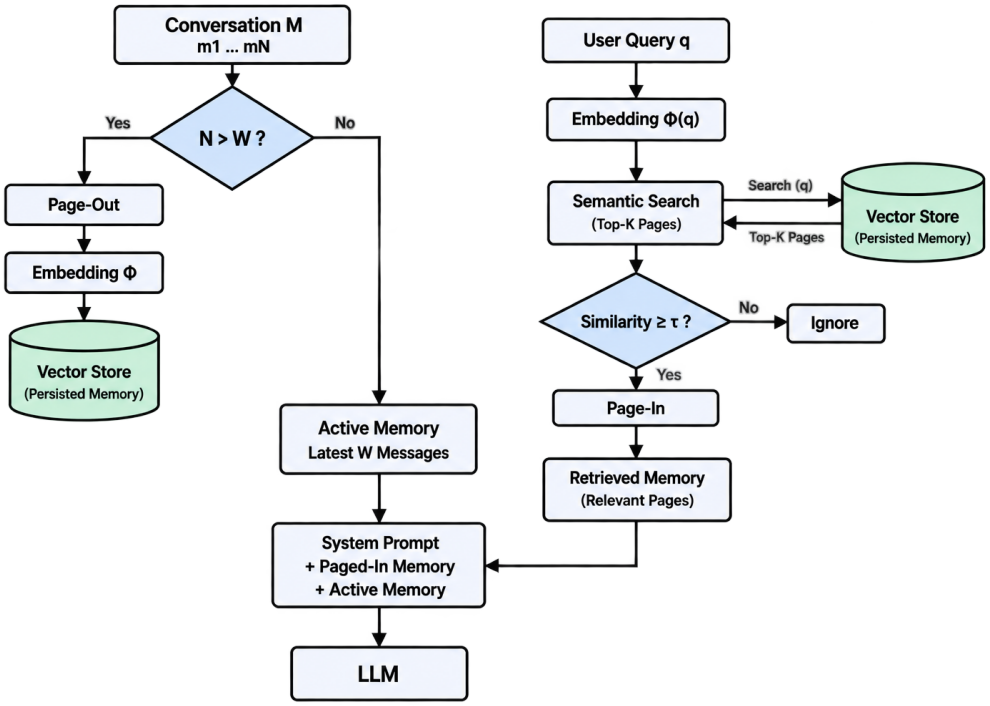


Figure 3.4: Context page-out, vector-store retrieval, and semantic page-in flow

3.6 GLOBAL SCHEDULER AND RESOURCE MANAGER

The scheduler maintains ready records and selects work globally

across active tasks. Effective priority is computed from base priority and queue waiting time, bounded by a maximum. Before dispatch, the resource manager checks the global slot limit, per-task slot limit, and token budget. The current defaults are three global execution slots, two slots per task, and a configurable 50,000-token task budget. Dispatch is cooperative: a running remote LLM request is not forcibly killed. Figure 3.5 shows the decision path.

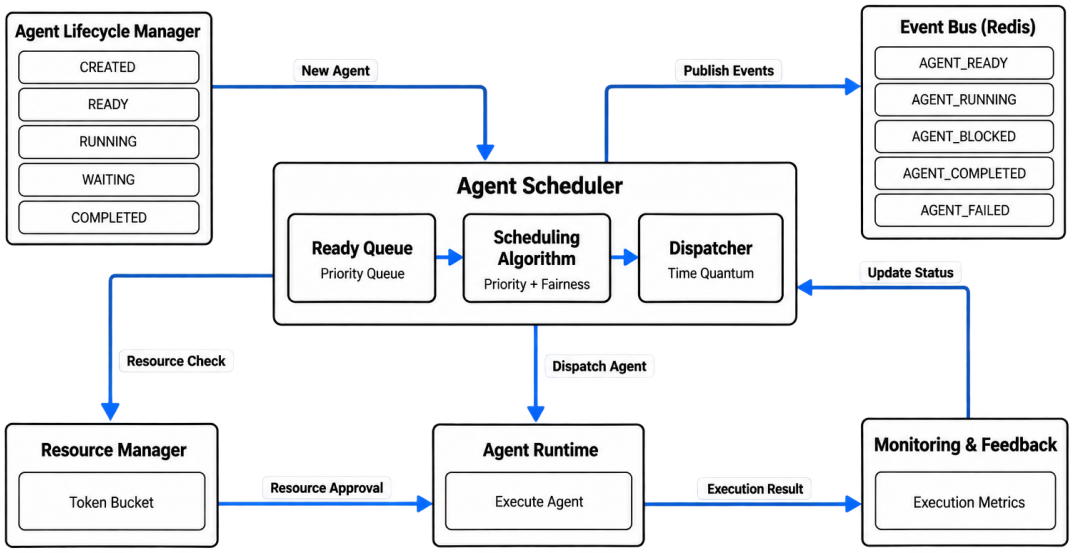


Figure 3.5: Global agent scheduler, lifecycle events, resource checks, and runtime feedback

3.7 RESOURCE MANAGEMENT

The resource-manager design in Figure 3.6 expands the admission point in the scheduler. It separates token-bucket accounting, quota monitoring, process monitoring, resource checking, and the final decision. In the current implementation, the corresponding resource manager enforces global and per-task execution slots and a token budget; the policy store, process monitor,

and hard preemption shown in the figure are design extensions.

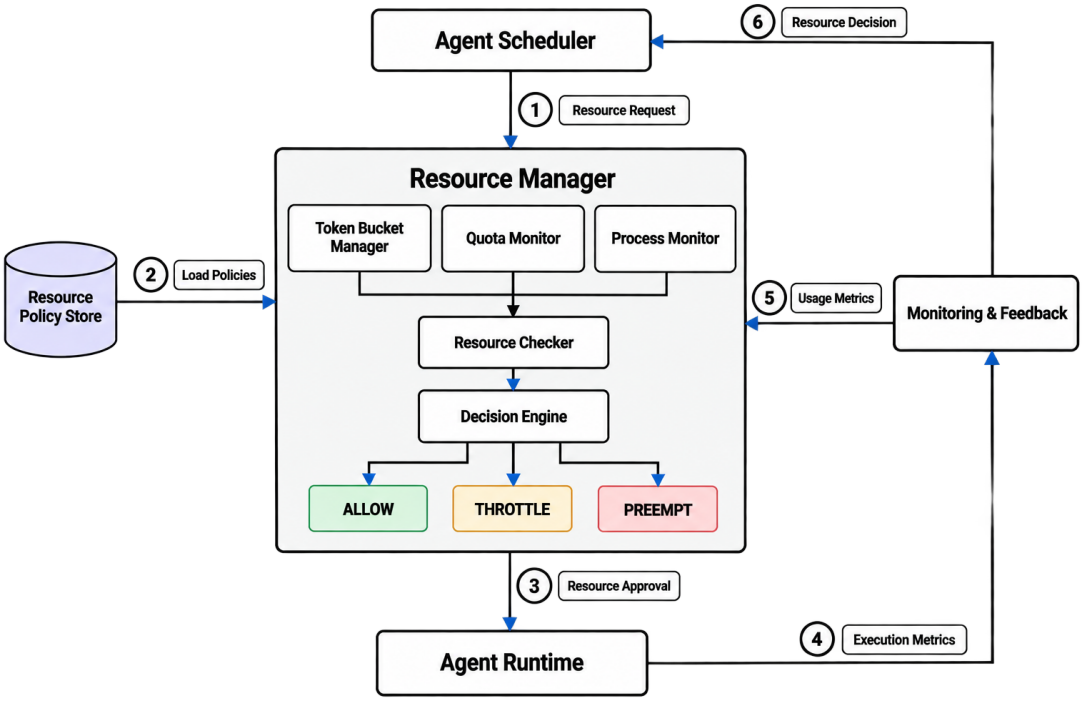


Figure 3.6: Resource-manager decision flow supplied for the AgentOS design

3.8 TOOL MANAGEMENT

Figure 3.7 shows the intended tool-manager boundary between an agent, a tool registry, permission policy, and local or external services. AgentOS currently exposes bounded tools through the runtime and code runner; a complete permission store and MCP-compatible tool manager remain proposed components.

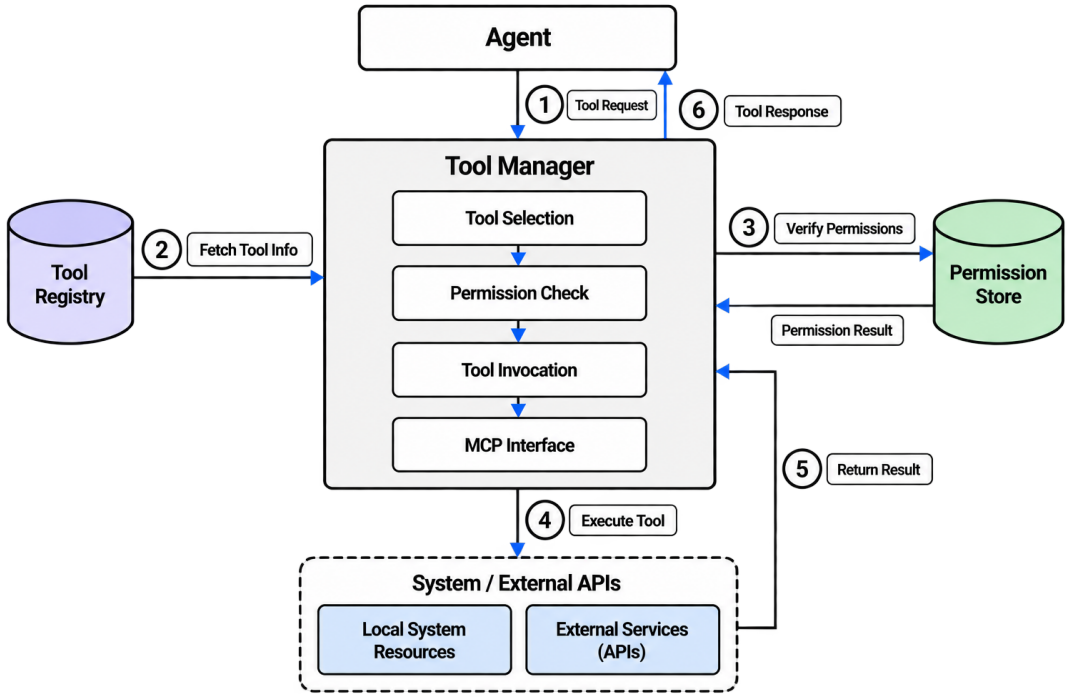


Figure 3.7: Tool selection, permission checking, invocation, and result return

3.9 AGENT LIFECYCLE AND EVENTS

An agent step progresses from CREATED to READY, RUNNING, and COMPLETED. It can move to WAITING when a resource or dependency is unavailable, or to BLOCKED/FAILED when execution cannot proceed. The scheduler and runtime publish structured lifecycle events containing task, step, agent, execution, status, priority, queue wait, and attempt information. Figure 3.8 shows the state model. RBAC, distributed mutex locking, and complete state-machine enforcement are architectural extensions rather than claims of full implementation in this phase.

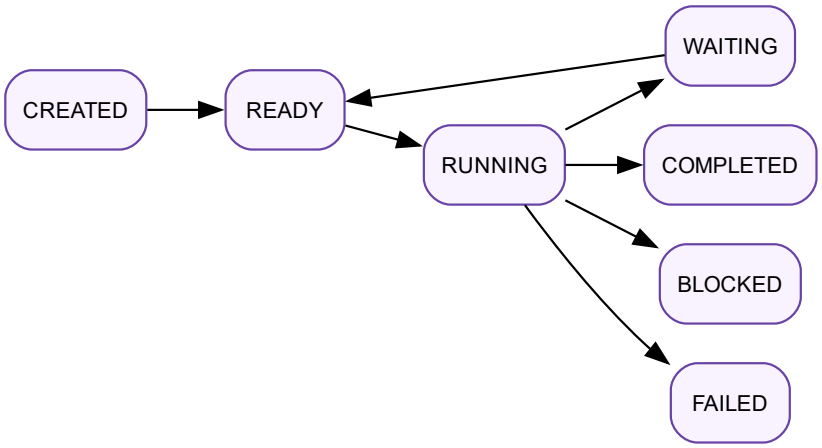


Figure 3.8: Agent lifecycle transitions

3.10 SANDBOXED CODE RUNNER

The code runner receives generated Python, writes it into a scoped working directory, and invokes it through an asynchronous subprocess. It bounds execution time and captured output. A non-zero exit code or traceback is returned to the coder/tester agent, which may produce a corrected version for at most three attempts. Output lines can be published as kernel events for live frontend observation. Figure 3.9 shows this flow. The current isolation is process and directory scoping; hardened container or microVM isolation is future work.

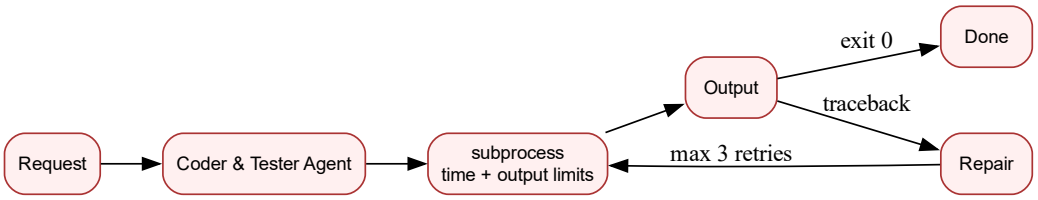


Figure 3.9: Sandboxed Python execution and bounded self-healing loop

3.11 IMPLEMENTED AND PROPOSED BOUNDARIES

Table 3.1 records the design boundary used throughout the report.

Table 3.1: Implemented and future AgentOS modules

Module	Current implementation	Status / limitation
Frontend and gateway	Next.js, FastAPI, WebSocket activity	Implemented
Planner and DAG	Structured plan, validation, Mermaid graph	Implemented
Scheduler and resources	Priority aging, slots, token admission	Implemented; cooperative only
Context paging	ChromaDB semantic swap and retrieval	Implemented
Redis and MongoDB	Events, queue metadata, task history	Implemented
Code runner	Bounded Python subprocess and retries	Implemented; not microVM isolation
RBAC / capability policy	Architecture documented	Proposed
Distributed locks / workers	Architecture documented	Future
Hybrid model routing	Provider abstraction is partial	Future
Durable coroutine restoration	Interrupted records are detected/requeued policy is partial	Future

CHAPTER 4

IMPLEMENTATION

4.1 IMPLEMENTATION ENVIRONMENT

The kernel is implemented in Python using asynchronous services. The backend is FastAPI-based and the frontend is implemented with Next.js and TypeScript. Redis provides task/event channels and scheduler metadata; MongoDB stores task records and event history; ChromaDB provides persistent vector collections for planning and context swap. Provider-specific LLM usage is treated as optional metadata because some providers do not return reliable token counts.

4.2 SEMANTIC AGENT RETRIEVAL

Agent descriptions are embedded in a persistent ChromaDB collection. A task description is used as a query, and the top candidate records are returned with their distances and metadata. In the configured cosine space, ChromaDB returns $d = 1 - \cos(\theta)$, so the implementation reports $s = 1 - d$. It does not divide by two; that transformation would be appropriate for a different distance normalisation. An LRU cache reduces repeated retrieval for identical descriptions. Algorithm 4.1 summarises this candidate-retrieval and cosine-distance conversion procedure.

Algorithm 4.1 Semantic Agent Retrieval

Require: task description q, agent collection C, result count k

$r \leftarrow C.\text{query}(q,k)$
for all candidate distances d_i in r **do**
    $s_i \leftarrow 1 - d_i$
    attach $(agent_i, s_i, description_i)$ to candidate
end for
sort candidates by s_i descending
return top candidates

4.3 PLANNER AND FALLBACK

The planner asks the configured LLM for a structured execution plan. The response is parsed into steps and normalised against the agent registry. If the LLM request fails, exceeds quota, or produces invalid structure, a deterministic fallback creates a usable plan from the best retrieved agent(s). This maintains task execution and observability without pretending that the fallback has the same reasoning quality as the provider.

Figure 4.1 illustrates the complete optimized planning path, from semantic candidate retrieval through structured plan validation, fallback routing, DAG construction, scheduling, and runtime execution.

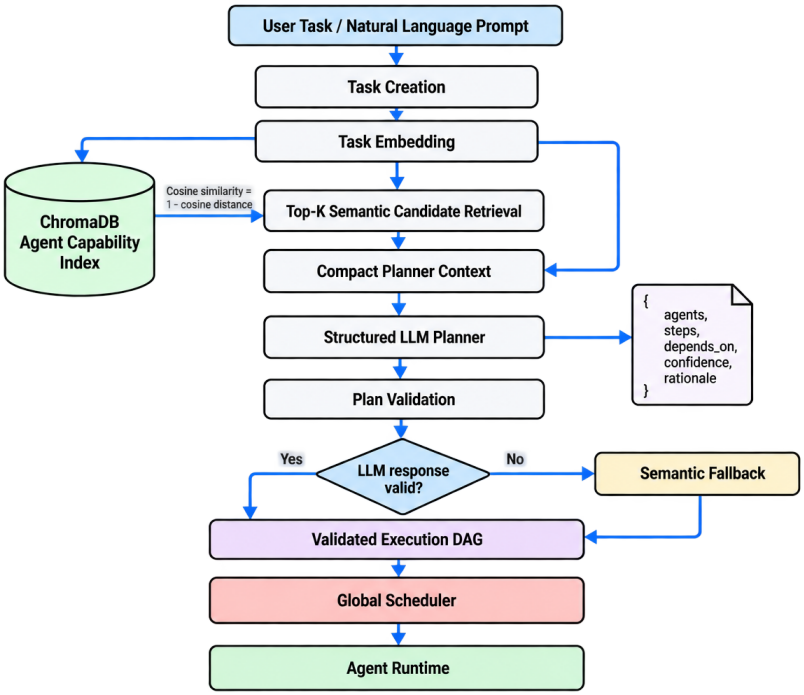


Figure 4.1: Optimized semantic planner and DAG construction

4.4 DAG VALIDATION

Every step has a unique identifier and a list of prerequisite identifiers. Validation rejects duplicate identifiers, unknown agents, unknown dependencies, and cycles. A Kahn-style topological pass counts nodes removed from the ready set. If the count is smaller than the number of steps, the dependency graph contains a cycle and is rejected. The same validated dependency lists are used to generate Mermaid edges. Algorithm 4.2 specifies the cycle-detection procedure.

Algorithm 4.2 Dependency Validation by Cycle Detection

Require: step set S
build adjacency list and indegree for every step
$Q \leftarrow$ all steps with indegree zero
$visited \leftarrow 0$
while Q is not empty **do**
    remove a step u from Q; $visited \leftarrow visited + 1$
    for all successors v of u **do**
        decrement indegree(v)
        if indegree$(v) = 0$ **then** add v to Q
        end if
    end for
end while
if $visited \neq |S|$ **then return** invalid cyclic plan
elsereturn valid plan
end if

4.5 PRIORITY-AGING SCHEDULER

The scheduler stores ready records in a Redis sorted set and keeps execution metadata in hashes. The selection score is conceptually computed as:

$$P_{effective} = \min(P_{base} + \alpha W, P_{max}), \tag{4.1}$$

where W is queue waiting time. The implementation uses an in-process pending set for dispatch coordination and persists the record so that ready and waiting work is visible and recoverable. A selected record is marked RUNNING, dispatched once, and then completed, requeued, or failed according to the result. Algorithm 4.3 gives the corresponding cooperative scheduling procedure.

Algorithm 4.3 Global Priority-Aging Cooperative Scheduler

while scheduler is active **do**
    receive and persist READY records
    if ready queue is empty **then** wait for event; continue
    end if
    for all ready records a_i **do** calculate $P_{effective,i}$ using Equation 4.1
    end for
    $a^* \leftarrow \arg\max P_{effective,i}$
    if resource manager denies a^* temporarily **then** mark WAITING and requeue
    else if resource manager denies permanently **then** mark FAILED and publish event
    elsedispatch a^* cooperatively and publish RUNNING
        if execution completes **then** publish COMPLETED
        else if execution waits or is requeued **then** publish WAITING or PREEMPTED
        elsepublish FAILED
        end if
    end if
    release resource reservations and record metrics
end while

4.6 RESOURCE ADMISSION AND TOKEN BUDGETS

The resource manager checks active global slots, active slots for the task, and the cumulative estimated token reservation for the task. A temporary denial leaves work waiting; an exhausted task budget is converted into a blocked/failed outcome rather than leaving the task silently pending. Actual provider usage, when available, is recorded separately from estimated admission tokens. This distinction explains why a provider such as Groq may display zero usage metadata even while the scheduler correctly enforces its configured estimate.

4.7 REDIS EVENTS AND BACKEND FORWARDING

Events are JSON records containing task identifier, step identifier, agent name, execution identifier, status, timestamp, priority, effective priority, queue wait, and attempt. The kernel publishes them on `kernel_events`; the backend subscribes and forwards them over WebSocket. Scheduler records use keys such as `agentos:scheduler:ready` and `agentos:scheduler:job:<execution_id>`. MongoDB persists task history and event records for later inspection. Table 4.1 summarises the Redis channels and scheduler keys used for task transport, lifecycle events, queue persistence, and recovery.

Table 4.1: Principal Redis channels and keys

Name	Purpose	Durability role
`backend_tasks`	New task/submission messages	Transport
`kernel_events`	Lifecycle, scheduler, and execution events	Transport plus Mongo persistence
`agentos:scheduler:ready`	Ready queue sorted-set metadata	Recovery aid
`agentos:scheduler:job:<id>`	Per-execution state hash	Recovery aid
`agentos:task:<id>`	Task-level scheduler state	Inspection/recovery

4.8 CONTEXT PAGE-OUT AND PAGE-IN

The pager moves messages from the active conversation state to a persistent ChromaDB collection after a configured window is exceeded. Each page stores message content, task metadata, and an embedding. On a later turn, the current query is embedded and the collection is searched; selected messages are reintroduced in relevance order. The implementation emits page-out and page-in metrics, allowing the UI or logs to verify that paging occurred.

4.9 SANDBOXED EXECUTION AND SELF-HEALING

The code runner creates a scoped temporary script, launches it with `asyncio.create_subprocess_exec`, captures stdout/stderr, and terminates the subprocess after a strict timeout. Output size is limited to prevent unbounded memory use. On a traceback, the error is supplied to the coder/tester agent and a revised script is attempted, up to three times. The runner publishes line-oriented output and final status as kernel events. No claim is made that an ordinary subprocess is equivalent to a container or microVM security boundary.

4.10 FRONTEND OBSERVABILITY

The task activity panel displays lifecycle events, scheduler state, queue position, effective priority, waiting time, attempt number, resource decisions, and token totals when provider metadata is present. The plan graph modal displays the structured graph and Mermaid source. Token fields distinguish input, output, total, and configured budget; when the provider omits usage, the interface reports that the metadata is unavailable rather than inventing a count.

4.11 CONFIGURATION AND VERIFICATION

Table 4.2 lists the important policy values. Verification for this phase consists of Python compilation, TypeScript type checking, scheduler/resource smoke tests, DAG validation checks, and manual event/graph inspection. No throughput, latency, or model-quality benchmark is claimed because a controlled benchmark dataset and repeated measurement harness are not yet part of the repository.

Table 4.2: Selected AgentOS configuration values

Configuration	Default	Role
Global execution slots	3	Maximum simultaneously admitted jobs
Per-task execution slots	2	Fairness/isolation across tasks
Task token budget	50,000	Cumulative estimated admission limit
Maximum scheduler priority	configurable	Upper bound for aging
Context active window	configurable	Number of active messages
Self-healing attempts	3	Maximum code repair retries

4.12 AGENTOS ALGORITHMIC SPECIFICATION

The following algorithm sheets define the AgentOS execution design. Together they specify the relationship between the planner, executor, lifecycle manager, context pager, dependency-aware DAG executor, scheduler, and priority-aging model. They complement the executable implementation described above. The sheets that describe full execution-context restoration, hard time-slice suspension, permission stores, or preemption represent planned extensions where the current implementation remains cooperative or partial. Algorithms 4.1, 4.2, and 4.3 describe the implemented retrieval, validation, and scheduler procedures. Algorithms 4.4, 4.5, 4.6, 4.7, 4.8, and 4.9 present the lifecycle, context-paging, cooperative execution, semantic planning, DAG, and global scheduling specifications in that order. The separate priority-aging/time-slice model previously shown as Figure 4.7 has been removed; the implemented priority equation and scheduler procedure are documented earlier in this chapter.

Algorithm 4.4 Agent Lifecycle Management

Require: $agent_id$, $agent_type$, $task_id$
Ensure: Managed agent runtime

$runtime_store[agent_id] = (id, type, task_id, run_fn, status)$
$status \in \{\text{CREATED}, \text{RUNNING}, \text{COMPLETED}, \text{FAILED}\}$
function CREATEAGENT($agent_id, agent_type, task_id$)
    if $agent_id \in runtime_store$ **then**
        raise "Agent already exists"
    end if
    $run_fn \leftarrow$ GETAGENTRUNFUNCTION($agent_type$)
    $runtime_store[agent_id] \leftarrow (agent_id, agent_type, task_id, run_fn, \text{CREATED})$
    return $runtime_store[agent_id]$
end function
function SETAGENTSTATUS($agent_id, status$)
    $runtime_store[agent_id].status \leftarrow status$
end function
function DESTROYAGENT($agent_id$)
    DELETE(runtime_store[agent_id])
end function

Algorithm 4.5 Virtual Memory Context Paging

Require: T_{id}, query q, messages $M = \langle m_1, \ldots, m_N \rangle$, window W, threshold τ, top-K limit K, embedding function Φ
Ensure: C_{final}, the context supplied to the LLM

$C_{final} \leftarrow [\text{SystemMessage}(\text{SYSTEM_PROMPT})]$; $P_{in} \leftarrow \emptyset$
if $N > W$ **then**
  $P_{overflow} \leftarrow M[1 : N - W]$
  for all $m_i \in P_{overflow}$ **do**
    $page_key \leftarrow T_{id} \| \texttt{_msg_} \| i$
    if not EXISTS($page_key$) **then**
      $v_i \leftarrow \Phi(m_i.content)$
      $metadata \leftarrow \{T_{id}, m_i.role, m_i.timestamp, i\}$
      VECTORSTORE.INSERT($page_key, v_i, m_i.content, metadata$)
      EMITEVENT(`context.PAGE_OUT`, T_{id}, i)
    end if
  end for
  $M_{active} \leftarrow M[N - W + 1 : N]$
else
  $M_{active} \leftarrow M$
end if
if $N > W$ **and** $\text{Trim}(q) \neq \emptyset$ **then**
  $v_q \leftarrow \Phi(q)$; $Candidates \leftarrow$ VECTORSTORE.SEARCH(T_{id}, v_q, K)
  for all $p \in Candidates$ **do**
    $s(p) \leftarrow \dfrac{v_q \cdot v_p}{\|v_q\| \|v_p\|}$
    if $s(p) \geq \tau$ **then**
      $P_{in} \leftarrow P_{in} \cup \{p\}$
      EMITEVENT(`context.PAGE_IN`, $T_{id}, s(p), p.text$)
    end if
  end for
end if
if $P_{in} \neq \emptyset$ **then**
  $R \leftarrow$ FORMATPAGEDMEMORY(P_{in})
  $C_{final} \leftarrow C_{final} \cup \{\text{SystemMessage}(R)\}$
end if
for all $m \in M_{active}$ **do**
  if $m.role =$ `human` **then**
    C_{final}.APPEND(HumanMessage($m.content$))
  else if $m.role =$ `ai` **then**
    C_{final}.APPEND(AIMessage($m.content$))
  end if
end for
return C_{final}

Algorithm 4.6 Cooperative Time-Sliced Agent Execution

Require: Agent A^* and time quantum T_q
Ensure: Status in $\{\text{COMPLETED}, \text{WAITING}, \text{PREEMPTED}, \text{FAILED}\}$

procedure EXECUTEAGENT(A^*, T_q)
  if $A^*.execution_context$ exists **then**
    $runtime \leftarrow$ RESTOREEXECUTIONCONTEXT(A^*)
  else
    $runtime \leftarrow$ INITIALIZEAGENTRUNTIME(A^*)
  end if
  $start_time \leftarrow$ GETCURRENTTIME; SETAGENTSTATUS($A^*.id$, RUNNING)
  while true do
    if AGENTEXECUTIONFAILED(runtime) **then**
      SAVEEXECUTIONCONTEXT($A^*, runtime$); **return** FAILED
    end if
    if AGENTISWAITING(runtime) **then**
      SAVEEXECUTIONCONTEXT($A^*, runtime$); **return** WAITING
    end if
    if GETCURRENTTIME $- start_time \geq T_q$ **then**
      SAVEEXECUTIONCONTEXT($A^*, runtime$); SUSPENDAGENTEXECUTION(runtime); **return** PREEMPTED
    end if
    $result \leftarrow$ EXECUTENEXTSTEP($runtime$)
    if $result =$ COMPLETED **then**
      $A^*.result \leftarrow$ GETEXECUTIONRESULT($runtime$); DESTROYEXECUTIONCONTEXT(runtime); **return** COMPLETED
    else if $result =$ WAITING **then**
      SAVEEXECUTIONCONTEXT($A^*, runtime$); **return** WAITING
    else if $result =$ FAILED **then**
      SAVEEXECUTIONCONTEXT($A^*, runtime$); **return** FAILED
    end if
  end while
end procedure

Algorithm 4.7 Semantic Dependency-Aware Agent Planning

Require: User task q, registry A, top-K limit K, planner model L
Ensure: Valid execution DAG $G = (V, E)$

if capability index is not initialized **then**
  for all $a_i \in A$ **do**
    $d_i \leftarrow$ capability description; $e_i \leftarrow \text{Embed}(d_i)$; store (a_i, d_i, e_i)
  end for
end if
$e_q \leftarrow \text{Embed}(q)$; $C \leftarrow \text{SelectTopKAgents}(A, \text{CosineSimilarity}(e_q, e_i), K)$
$P \leftarrow \text{BuildPlannerPrompt}(q, C)$; $R \leftarrow \text{LanguageModelGenerateJSON}(L, P)$
if R contains valid DAG steps **then**
  $S \leftarrow R.steps$
else
  $S \leftarrow \text{ConvertAgentListToSequentialSteps}(R.agents)$
end if
$S \leftarrow \text{RemoveUnknownAgents}(S, A)$
$S \leftarrow \text{RemoveAgentsNotInCandidates}(S, C)$
$S \leftarrow \text{RemoveDuplicateStepIDs}(S)$; $S \leftarrow \text{RemoveInvalidDependencies}(S)$
$S \leftarrow \text{ValidateAcyclicity}(S)$
if S contains a cycle **then**
  $S \leftarrow \text{ConvertAgentListToSequentialSteps}(R.agents)$
end if
$G \leftarrow \text{BuildGraph}(S)$; $M \leftarrow \text{GenerateMermaidDiagram}(G)$
$\text{Persist}(G, M)$; $\text{EmitPlanningEvent}(G, M)$
return G

Algorithm 4.8 Dependency-Aware Agent DAG Execution

Require: Execution DAG $G = (V, E)$ and task state
Ensure: Results produced by all executed agents

$completed \leftarrow \emptyset$
while $completed \neq V$ **do**
    $ready \leftarrow \emptyset$
    for all $v \in V$ **do**
        if $v \notin completed$ **and** Dependencies$(v) \subseteq completed$ **then**
            $ready \leftarrow ready \cup \{v\}$
        end if
    end for
    if $ready = \emptyset$ **then**
        raise InvalidPlanCycle
    end if
    $v \leftarrow$ SchedulerSelect$(ready)$; $result \leftarrow$ ExecuteAgent$(v.agent)$
    StoreResult$(v, result)$; $completed \leftarrow completed \cup \{v\}$
end while
return all results

Algorithm 4.9 Global Priority-Aging Cooperative Agent Scheduler

Require: Redis ready queue Q, aging factor α, maximum priority P_{max}, quantum T_q, resource manager R
Ensure: Scheduled agent execution

while AgentOS is active **do**
  Receive `AGENT_READY` events and insert agents into Q
  if $Q = \emptyset$ **then**
    Wait for the next scheduler event; **continue**
  end if
  $t \leftarrow \text{CurrentTime}()$
  for all $A_i \in Q$ **do**
    $W_i \leftarrow t - A_i.ready_time$; $P_i^{eff} \leftarrow \min(P_i^{base} + \alpha W_i, P_{max})$
  end for
  $A^* \leftarrow \arg\max_{A_i \in Q} P_i^{eff}$; $auth \leftarrow R.\text{request}(A^*)$
  if $auth =$ DENIED **then**
    if denial is temporary **then**
      Mark A^* WAITING and reinsert into Q
    else
      Mark A^* FAILED and publish `AGENT_FAILED`
    end if
    continue
  end if
  Remove A^* from Q; mark RUNNING; publish `AGENT_RUNNING`
  $r \leftarrow \text{Dispatch}(A^*, T_q)$
  if $r =$ COMPLETED **then**
    Mark COMPLETED; publish `AGENT_COMPLETED`
  else if $r =$ WAITING **then**
    Mark WAITING
  else if $r =$ PREEMPTED **then**
    Mark READY, update *ready_time*, reinsert into Q, publish `AGENT_PREEMPTED`
  else
    Mark FAILED; publish `AGENT_FAILED`
  end if
  ReleaseResources(A^*); RecordMetrics(A^*, r)
end while

CHAPTER 5

EVALUATION FRAMEWORK AND METRICS

This appendix defines the metrics that will be used to evaluate AgentOS. The metrics specify how planning quality, execution reliability, scheduling behaviour, resource control, context paging, event delivery, and user-facing observability will be measured. No experimental values are reported here; the definitions are intended to support a later controlled evaluation.

5.1 EVALUATION NOTATION

Let N be the number of evaluated tasks, S the number of planned steps, and E the number of emitted events. For each task, the expected result, required agents, dependency relationships, and final status must be recorded before execution. A task is counted as successful only when its required output is produced and all mandatory steps complete without an unrecovered error.

5.2 PLANNING QUALITY METRICS

Table 5.1 defines the planning-quality measures used to assess agent selection, plan validity, and dependency correctness.

Table 5.1: Metrics for evaluating planner quality

Metric	Definition and formula
Agent-selection accuracy	Fraction of tasks for which the planner selects the complete required agent set: $\frac{\text{tasks with correct agent set}}{N}$.
Agent precision	Fraction of selected agents that are relevant: $\frac{TP}{TP+FP}$.
Agent recall	Fraction of required agents that are selected: $\frac{TP}{TP+FN}$.
Agent F1-score	Harmonic mean of selection precision and recall: $F_1 = 2\frac{PR}{P+R}$.
Plan validity rate	Fraction of generated plans that pass schema, agent-identifier, dependency, and cycle validation: $\frac{\text{valid plans}}{\text{generated plans}}$.
Dependency correctness	Fraction of dependency relations that match the expected relations: $\frac{\text{correct dependency edges}}{\text{expected dependency edges}}$.
Fallback rate	Fraction of tasks executed using semantic fallback: $\frac{\text{fallback tasks}}{N}$.

For benchmark tasks with more than one acceptable plan, agent-selection accuracy should use set-based comparison and dependency correctness should use the union of all accepted dependency graphs.

5.3 EXECUTION RELIABILITY METRICS

Table 5.2 defines the reliability measures used to assess task completion, failures, retries, recovery, ordering, and latency.

Table 5.2: Metrics for execution reliability

Metric	Definition and formula
Task success rate	$\frac{\text{tasks completed with the expected result}}{N}$.
Step success rate	$\frac{\text{successfully completed steps}}{\text{dispatched steps}}$.
Failure rate	$\frac{\text{tasks ending in FAILED}}{N}$, reported separately from temporary WAITING or BLOCKED states.
Retry rate	$\frac{\text{steps executed more than once}}{\text{dispatched steps}}$.
Recovery correctness	$\frac{\text{interrupted steps safely requeued without duplicate completion}}{\text{interrupted steps requiring recovery}}$.
Dependency-order violation rate	$\frac{\text{steps started before all prerequisites completed}}{\text{started steps}}$. The desired value is zero.
End-to-end task latency	$T_{complete} - T_{created}$, reported using mean, median, and percentile values such as P_{95}.

5.4 SCHEDULER AND RESOURCE METRICS

Table 5.3 defines the scheduler and resource measures used to assess waiting time, fairness, concurrency, denial handling, and token-budget compliance.

Table 5.3: Metrics for scheduling and resource admission

Metric	Definition and formula
Mean queue waiting time	$\frac{1}{S}\sum_{i=1}^{S}(T_{start,i} - T_{ready,i})$.
Scheduling response time	$T_{dispatch} - T_{ready}$ for each admitted step.
Starvation rate	$\frac{\text{ready steps exceeding the starvation threshold}}{\text{ready steps}}$.
Fairness across tasks	Jain's fairness index: $J = \frac{(\sum_i x_i)^2}{n\sum_i x_i^2}$, where x_i is the service received by task i.
Concurrency-limit violations	$\frac{\text{dispatches exceeding global or per-task limits}}{\text{dispatches}}$. The desired value is zero.
Temporary-denial handling rate	$\frac{\text{temporarily denied jobs correctly waiting and requeued}}{\text{temporary denials}}$.
Token-budget compliance	$\frac{\text{tasks whose admitted estimate remains within budget}}{N}$.
Token utilisation	$\frac{\text{consumed or reported tokens}}{\text{configured token budget}}$, reported separately for estimated and provider-reported usage.

5.5 CONTEXT PAGING AND RETRIEVAL METRICS

The context-paging experiment should compare a bounded-context run with a run in which the complete conversation is available.

- Page-out rate $= \frac{\text{messages paged out}}{\text{messages received}}$.
- Page-in precision@$k = \frac{\text{relevant pages among top-}k\text{ retrieved pages}}{k}$.
- Page-in recall $= \frac{\text{relevant pages retrieved}}{\text{relevant pages required}}$.
- Context-retrieval mean similarity $= \frac{1}{m}\sum_{j=1}^{m} s_j$, where $s_j = 1 - d_j$ and d_j is the ChromaDB cosine distance.
- Context token reduction $= 1 - \frac{\text{active-context tokens with paging}}{\text{full-context tokens}}$.
- Paging latency $= T_{page\text{-}in} - T_{query}$.

5.6 EVENT AND OBSERVABILITY METRICS

Redis and WebSocket behaviour will be evaluated by correlating the event identifier and execution identifier in Redis, MongoDB, the backend stream, and the frontend activity log.

- Event delivery rate $= \frac{\text{events received by the intended consumer}}{E}$.
- Event loss rate $= 1 -$ event delivery rate.
- Event duplication rate $= \frac{\text{duplicate event records}}{E}$.
- Event ordering accuracy $= \frac{\text{lifecycle transitions observed in valid order}}{\text{observed lifecycle transitions}}$.
- WebSocket propagation latency $= T_{frontend\ receives} - T_{kernel\ publishes}$.
- Observability completeness $= \frac{\text{events containing all required fields}}{E}$.

The required event fields are task identifier, step identifier, agent name, execution identifier, status, timestamp, priority, effective priority, queue wait, and attempt number.

5.7 CODE RUNNER SAFETY METRICS

- Timeout enforcement rate $= \frac{\text{timed-out processes terminated within the policy window}}{\text{timed-out processes}}$.
- Output-capture completeness $= \frac{\text{runs with correctly captured stdout and stderr}}{\text{runs}}$.
- Self-healing repair success $= \frac{\text{failed scripts repaired within the retry limit}}{\text{failed scripts eligible for repair}}$.
- Retry exhaustion rate $= \frac{\text{scripts still failing after the maximum retries}}{\text{failed scripts}}$.
- Working-directory escape count: number of executions writing outside the configured working directory. The desired value is zero.

5.8 REPORTING PROTOCOL

The evaluation shall use a fixed task set and a documented procedure for every configuration.

1. **Define and repeat the test.** Record the prompts, expected outputs, required agents, dependency relations, configuration, number of tasks, and number of independent repetitions. Keep the planner model, candidate limit, context window, scheduler limits, resource limits, and retry policy fixed within a comparison.

2. **Record raw evidence.** Store the task identifier, prompt, expected result, candidates, generated plan, validation result, lifecycle events, attempts, timestamps, resource decisions, token metadata, and final status. Retain raw events so metrics can be recomputed.

3. **Compute task-level outcomes.** Mark a trial as successful only when the expected result is produced and all mandatory steps complete. Record whether the trial failed, waited, was retried, used fallback planning, violated dependency order, or exceeded a resource limit.

4. **Aggregate and report.** Report accuracy, success, failure, retry, fallback, and violation measures as percentages. For latency, queue waiting time, token usage, and other continuous measurements, report the mean, median, standard deviation, and 95th percentile (P_{95}), together with the number of trials and tasks.

   For a rate, use the pooled value $100 \times (\sum \text{numerator} / \sum \text{denominator})$ and state how missing or timed-out observations were handled. A zero denominator must be reported as *not applicable*, not as zero.

5. **Document annotations and usage.** Explain labels for expected agents or relevant context pages, including the annotation procedure

and number of labelled tasks. Distinguish provider-reported token usage from estimated admission usage.

Each configuration summary should therefore include task and repetition counts, success or accuracy rate, failure and retry rates, latency statistics, and relevant scheduler, resource, context-paging, and token-usage measures. No experimental values are reported in this appendix.

CHAPTER 6

SUMMARY AND FUTURE WORK

AgentOS provides a modular runtime for multi-agent applications in which planning, semantic agent selection, DAG execution, context management, scheduling, resource admission, sandboxed code execution, and frontend observability are treated as shared system services. The completed phase connects the gateway, planner, kernel, Redis event path, MongoDB task history, ChromaDB retrieval, scheduler, resource manager, agent runtime, and frontend activity interface into one execution pipeline.

The implementation is also bounded by explicit limitations. Scheduler dispatch is cooperative, token usage can be unavailable when a provider omits metadata, and restart recovery identifies interrupted work without restoring every active coroutine. The evaluation metrics defined in Appendix 5 provide the basis for measuring planning quality, execution reliability, scheduling fairness, resource compliance, context retrieval, event delivery, and code-runner safety; numerical results are reserved for controlled experiments.

Future work will therefore focus on durable restart resumption, true checkpoints inside long-running agents, complete capability enforcement, distributed scheduler workers, a hybrid local/cloud model router, cost-aware scheduling, formal workload benchmarks, and additional autonomous event triggers. These extensions can be added without changing the central separation between planner, scheduler, runtime, persistence, and observability layers.

CHAPTER 7

AGENT CATALOGUE AND INTERNAL WORKFLOWS

This appendix documents the agent implementations in `server/kernel/Agents/agents/`. The diagrams describe the internal responsibility of each agent and its actual tool or model interaction. The planner, scheduler, global execution state, and frontend are intentionally not shown here because they are documented in the system-design chapter.

Agent	Implemented responsibility
A1	System information and resource inspection.
A2	Date, time, and timezone queries.
A3	Factual and academic research with sources.
A4	File and document operations with deletion confirmation.
Content Creator	Text drafting, rewriting, and formatting.
Email	Email dispatch and delivery.
Weather	Current weather and forecast retrieval.
Loan EMI	EMI and amortisation calculation.
Crypto PnL	Cryptocurrency trade PnL and ROI reporting.
Code Runner	Python generation, execution, capture, and repair.
Agent Creator	Creation and registration of new specialised agents.

Figure 7.1: AgentOS agent catalogue

7.1 Agent A1: System Information Agent

The A1 implementation creates a LangChain agent with the single registered tool `get_system_info`. The tool reports the operating-system name and release, machine architecture, and processor descriptor using Python's `platform` module. The agent therefore provides a compact host identity report; process enumeration and detailed resource monitoring are not implemented by the current tool.

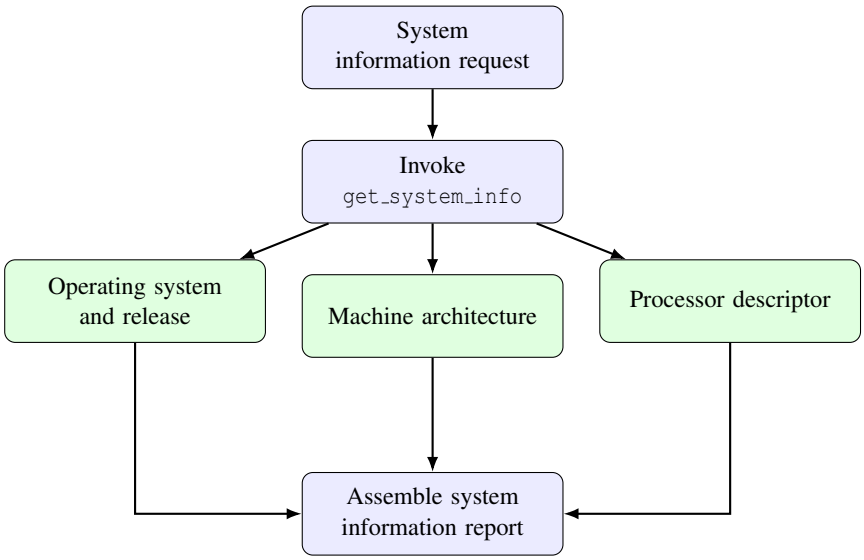


Figure 7.2: A1 system-information logic

7.2 Agent A2: Date and Time Agent

The A2 implementation creates a LangChain agent with `get_time`. The tool returns the host's current `datetime.now()` value as a formatted string. It does not currently accept an explicit timezone or location, so its answer represents the runtime host clock.

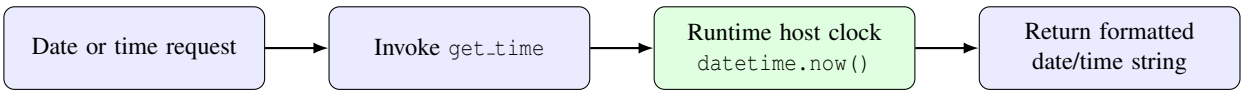


Figure 7.3: A2 date-and-time logic

7.3 Agent A3: Research Agent

The A3 implementation uses two tools: `wikipedia_search` calls the Wikipedia search and summary APIs and returns article titles, extracts, and URLs; `trusted_web_search` restricts search results to configured academic and scientific domains, validates result URLs, removes duplicates, and returns source snippets. The agent prompt requires concise evidence and prohibits fabricated citations or URLs.

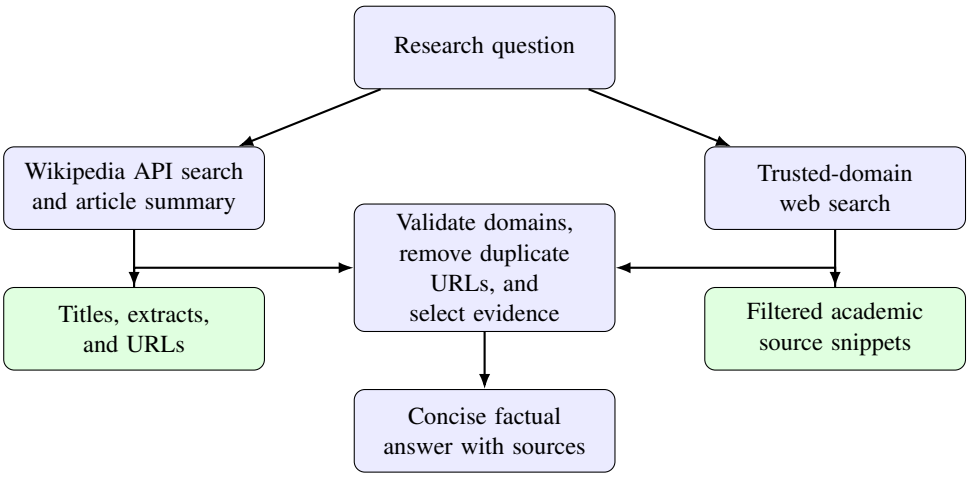


Figure 7.4: A3 research and source-verification logic

7.4 Agent A4: File and Document Agent

The A4 implementation exposes file discovery, bounded document reading, TXT/DOCX creation, append and exact replacement, folder creation, rename, move, metadata, directory listing, document comparison, and heuristic classification. PDF and DOCX readers are supported through `pypdf` and `python-docx`. There is deliberately no direct delete tool: `request_delete_confirmation` records a fingerprint in Redis, publishes a frontend confirmation event, and uses a 300-second expiry before the backend performs or cancels the operation.

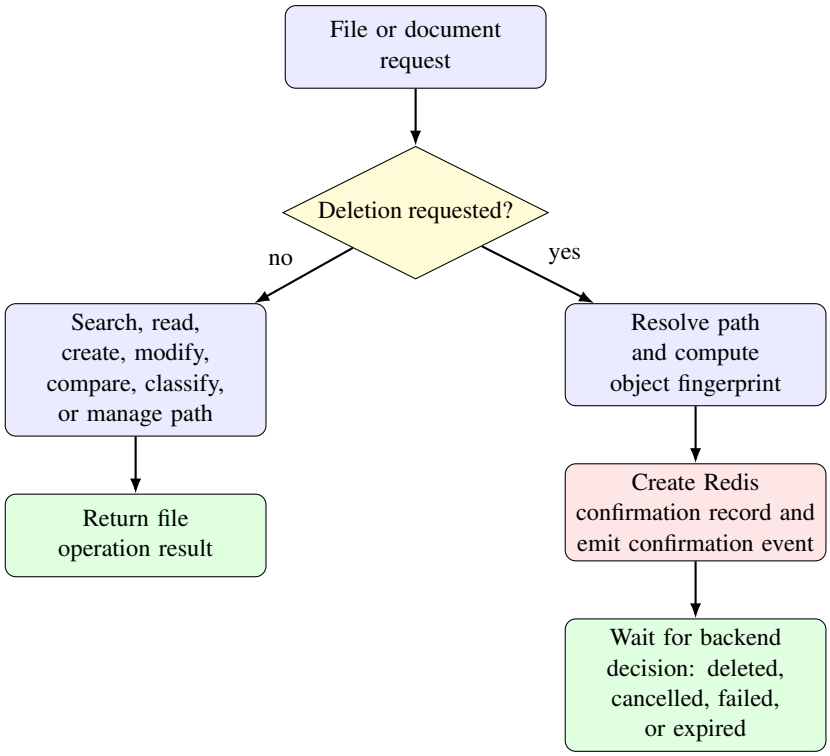


Figure 7.5: A4 file-operation and deletion-confirmation logic

7.5 Content Creator Agent

The Content Creator implementation creates a tool-free LangChain agent. It loads the current user name and email for sender context, selects the requested genre, and drafts, rewrites, summarises, or formats text. It is intentionally not an email-delivery component and must not send messages or emit unresolved placeholders.

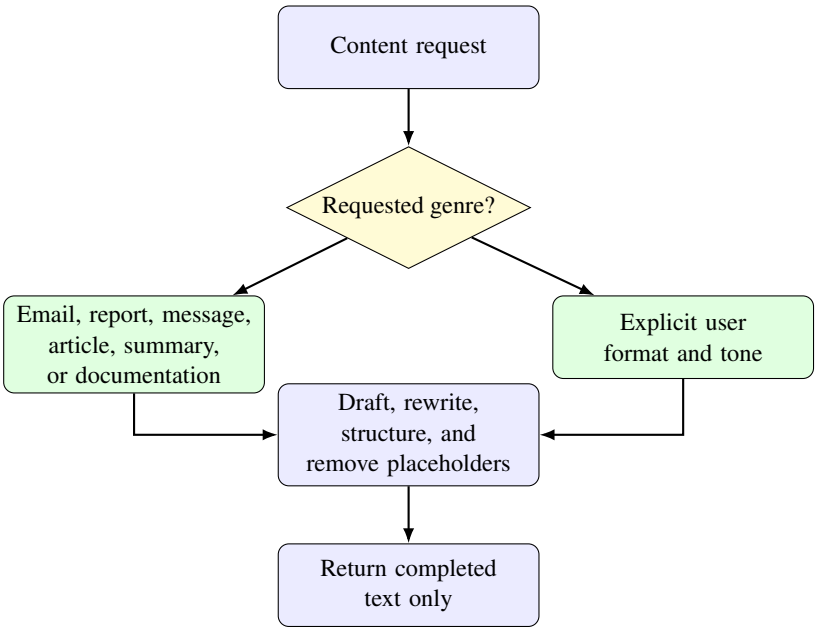


Figure 7.6: Content Creator drafting logic

7.6 Email Agent

The Email Agent uses `send_email`. The tool reads SMTP host, port, username, password, TLS, and sender settings from the environment. It sends through SMTP when credentials are available; otherwise it returns an explicit simulation-mode response and does not send network mail. The agent extracts the recipient, subject, and body, removes unresolved placeholders, and returns the tool's delivery status.

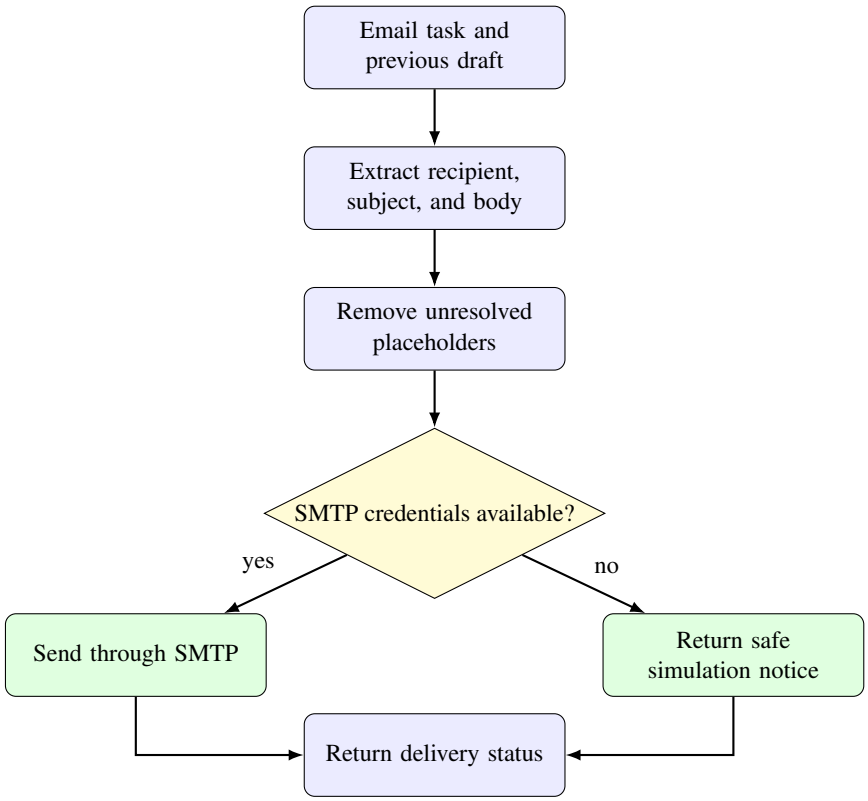


Figure 7.7: Email delivery and simulation-mode logic

7.7 Weather Agent

The Weather Agent uses Open-Meteo through `get_current_weather` and `get_weather_forecast`. Each operation geocodes the city first, then requests current or daily forecast data, maps WMO weather codes to readable conditions, and reports Celsius/Fahrenheit temperatures, humidity, wind, and precipitation. Forecast length is bounded to one through seven days.

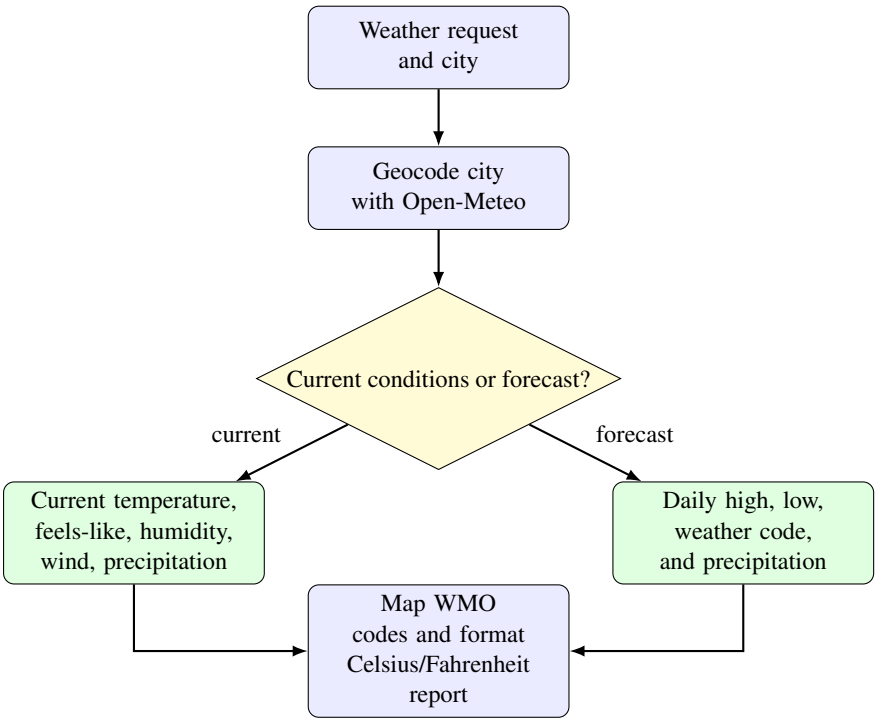


Figure 7.8: Weather retrieval and reporting logic

7.8 Loan EMI Calculator Agent

The Loan EMI Agent uses `compute_emi_and_schedule`. It converts the annual percentage rate to a monthly rate, computes the fixed-payment EMI, derives total payment and interest, and produces a monthly schedule for short tenures or an annual amortisation breakdown for longer tenures.

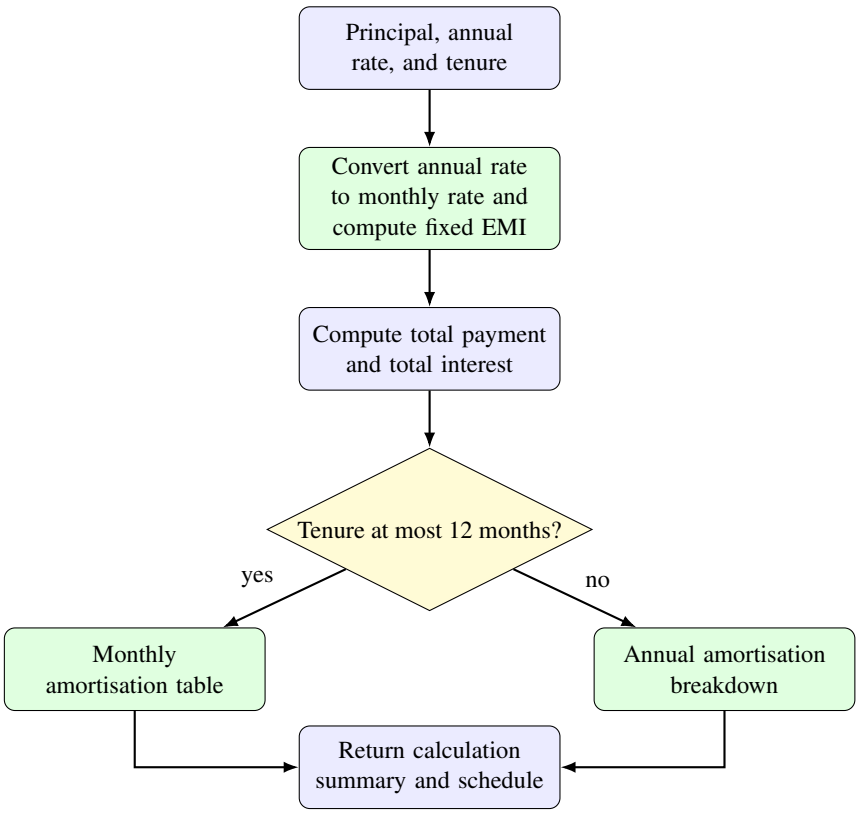


Figure 7.9: Loan EMI and amortisation logic

7.9 Crypto PnL Agent

The Crypto PnL Agent uses `calculate_crypto_pnl`. It calculates buy and sell notional values, applies the fee percentage to both sides, derives net profit, and computes ROI relative to total purchase outflow.

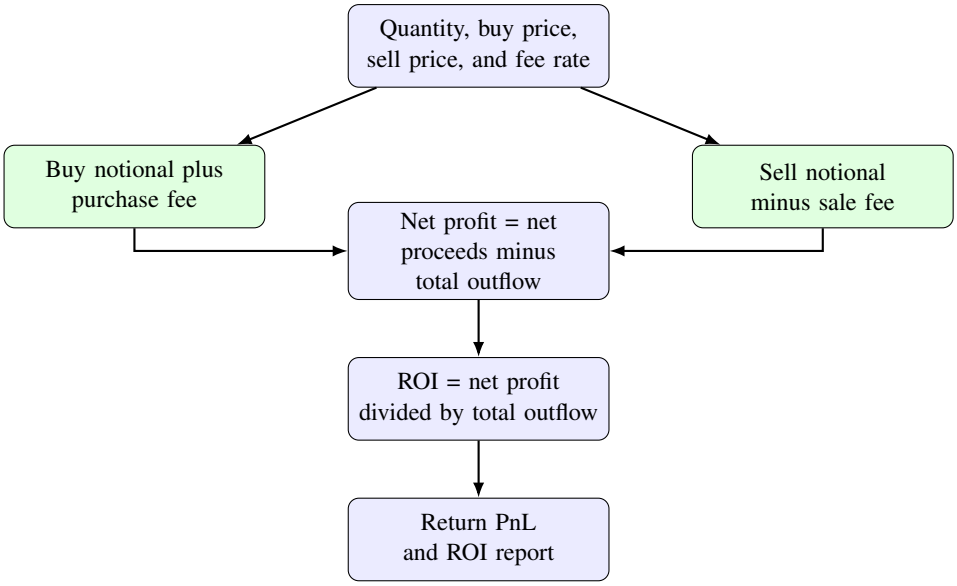


Figure 7.10: Cryptocurrency PnL and ROI logic

7.10 Code Runner Agent

The Code Runner Agent uses `build_run_python_code_tool`. It limits source code to 20,000 characters, executes Python with isolated mode and a sanitised environment, bounds runtime to ten seconds, captures at most 64,000 output characters, publishes terminal events, and limits concurrent runs to two. The agent repairs failures for at most three attempts.

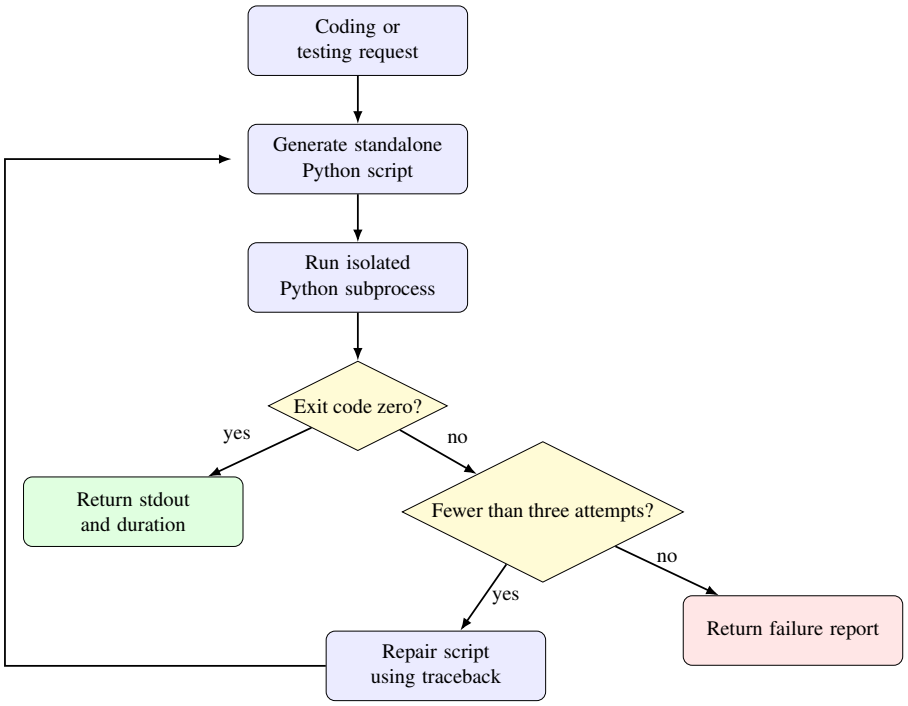


Figure 7.11: Code generation, sandbox execution, and repair logic

7.11 Agent Creator

The Agent Creator implementation is responsible for unsupported capabilities. `save_tool_file` parses generated source with the Python AST before writing and registering it. `create_and_register_agent` generates the agent module, updates the agent registry and planner constants, validates the changes, and rolls them back on failure. `run_generated_agent` then loads and executes the new agent. The implementation is present in the source directory but is commented out of the default registry.

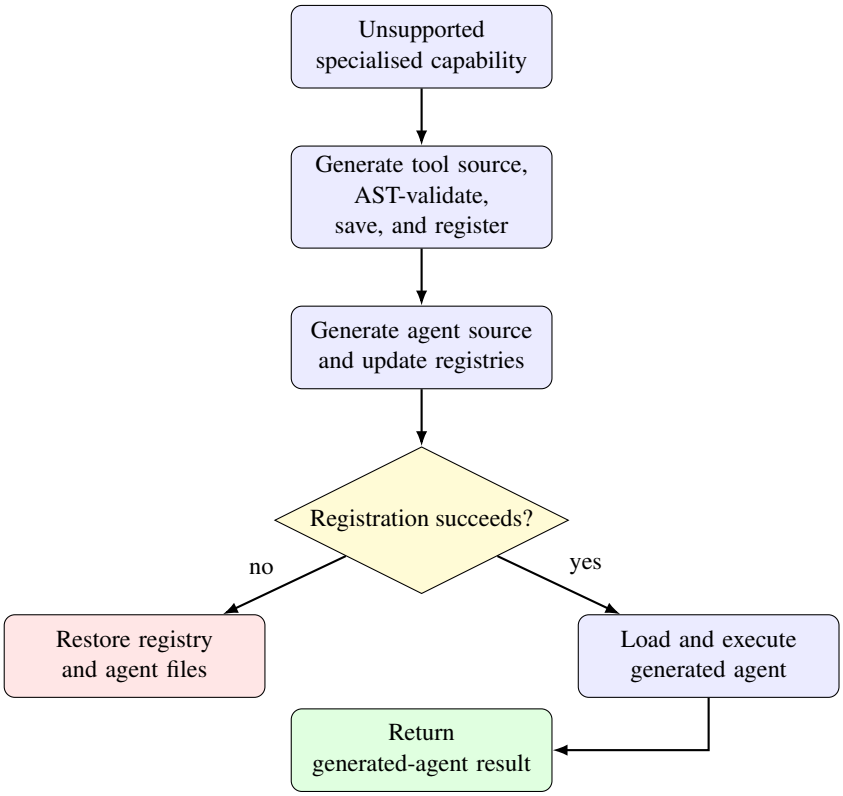


Figure 7.12: Dynamic agent creation and registration logic

REFERENCES

[1] K. Mei, X. Zhu, W. Xu, W. Hua, M. Jin, Z. Li, S. Xu, R. Ye, Y. Ge, and Y. Zhang, "AIOS: LLM Agent Operating System," arXiv:2403.16971, 2024.

[2] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J.-S. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A Survey on Large Language Model Based Autonomous Agents," *Frontiers of Computer Science*, vol. 18, no. 6, 2024, doi: 10.1007/s11704-024-40231-1.

[3] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone, "LLM+P: Empowering Large Language Models With Optimal Planning Proficiency," arXiv:2304.11477, 2023.

[4] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv:2308.08155, 2023.

[5] LangChain, "LangGraph Documentation," 2026. [Online]. Available: `https://langchain-ai.github.io/langgraph/` [Accessed: Oct. 6, 2026].

[6] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," arXiv:2310.08560, 2023.

[7] J. Kang, M. Ji, Z. Zhao, and T. Bai, "Memory OS of AI Agent," in *Proc. 2025 Conf. Empirical Methods in Natural Language Processing*, 2025, pp. 25961–25970, doi: 10.18653/v1/2025.emnlp-main.1318.

[8] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[9] Chroma, "Chroma Documentation: Collections and Configuration," 2026. [Online]. Available: `https://docs.trychroma.com/` [Accessed: Oct. 6, 2026].

[10] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. Int. Conf. Learning Representations*, 2023.

[11] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[12] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative Agents: Interactive Simulacra of Human Behavior," in *Proc. 36th ACM Symp. User Interface Software and Technology*, 2023, doi: 10.1145/3586183.3606763.

[13] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents With Verbal Reinforcement Learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[14] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[15] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[16] Linux Kernel Documentation, "Control Group v2," 2026. [Online]. Available: https://docs.kernel.org/admin-guide/cgroup-v2.html [Accessed: Oct. 6, 2026].

[17] LangChain, "LangGraph Persistence and Checkpointing," 2026. [Online]. Available: https://langchain-ai.github.io/langgraphjs/concepts/persistence/ [Accessed: Oct. 6, 2026].

[18] Temporal Technologies, "Temporal Documentation: Durable Execution," 2026. [Online]. Available: https://docs.temporal.io/ [Accessed: Oct. 6, 2026].

[19] Model Context Protocol, "Model Context Protocol Specification," 2026. [Online]. Available: https://modelcontextprotocol.io/specification/ [Accessed: Oct. 6, 2026].

[20] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," 2025. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/ [Accessed: Oct. 6, 2026].

[21] Amazon Web Services, "Firecracker MicroVM Documentation," 2026. [Online]. Available: https://github.com/firecracker-microvm/firecracker [Accessed: Oct. 6, 2026].

[22] Redis Ltd., "Redis Pub/Sub Documentation," 2026. [Online]. Available: https://redis.io/docs/latest/develop/interact/pubsub/ [Accessed: Oct. 6, 2026].

[23] MongoDB Inc., "MongoDB Documentation," 2026. [Online]. Available: https://www.mongodb.com/docs/ [Accessed: Oct. 6, 2026].